ECE PROJECT

Adversarially Robust I-Bug Navigation

Minimalist robot navigation under intermittent false-tower signals

Avraham Guez | Tsuriel Vizel
Supervised by Andrei Zhitnikov

Based on the 2014 paper by K. Taylor & S. M. LaValle

CRML
Control Robotics and Machine Learning Laboratory

Presentation Roadmap



BASELINE

Original I-Bug

THREAT

Adversarial tower

EXTENSIONS

Three strategies

IMPLEMENTATION

Julia simulator

EVALUATION

100,000 environments

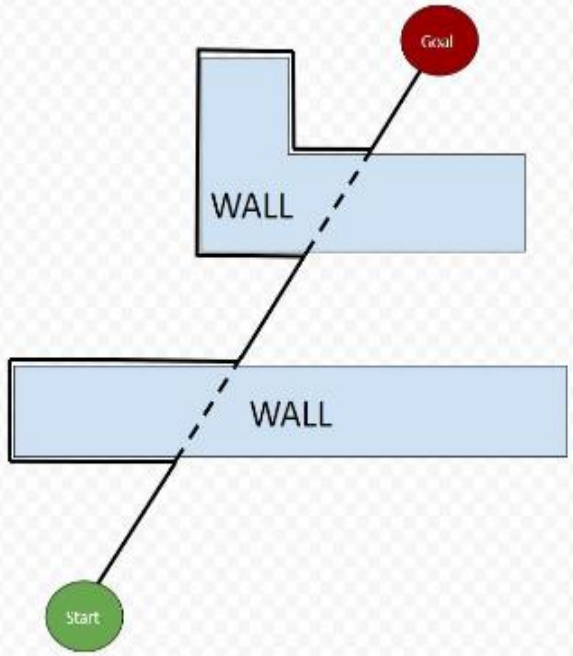
Storyline: assumption → attack → controller response → evidence

The Bug Algorithm Family

Bug algorithms draw inspiration from how simple insects navigate using basic behaviors like moving towards a goal and following obstacles when blocked.

- **Bug1 & Bug2:** The classic algorithms. They alternate between walking straight toward the target and circumnavigating blocking obstacles.
- **TangentBug:** Uses a limited range sensor to avoid hugging walls unnecessarily, cutting corners when a clear path is visible.

- **The Goal:** To guarantee reaching a target in a completely unknown environment with zero pre-loaded maps.



Why Minimalist Robots?



Lower Cost

By relying on extremely simple sensors rather than expensive LIDARs, high-end cameras, and powerful onboard computers, we drastically reduce hardware costs and complexity.



High Robustness

Fewer complex systems mean fewer points of failure. Minimalist systems consume significantly less energy, require less processing, and suffer from fewer integration issues.



Theoretical Limits

Academically, studying minimalism helps us prove the absolute baseline of information required to achieve complex tasks, ensuring the algorithms are mathematically sound.

Motivation: Navigation with Minimal Information

Reduce sensing and computation without giving up obstacle avoidance

NO GLOBAL LOCALIZATION

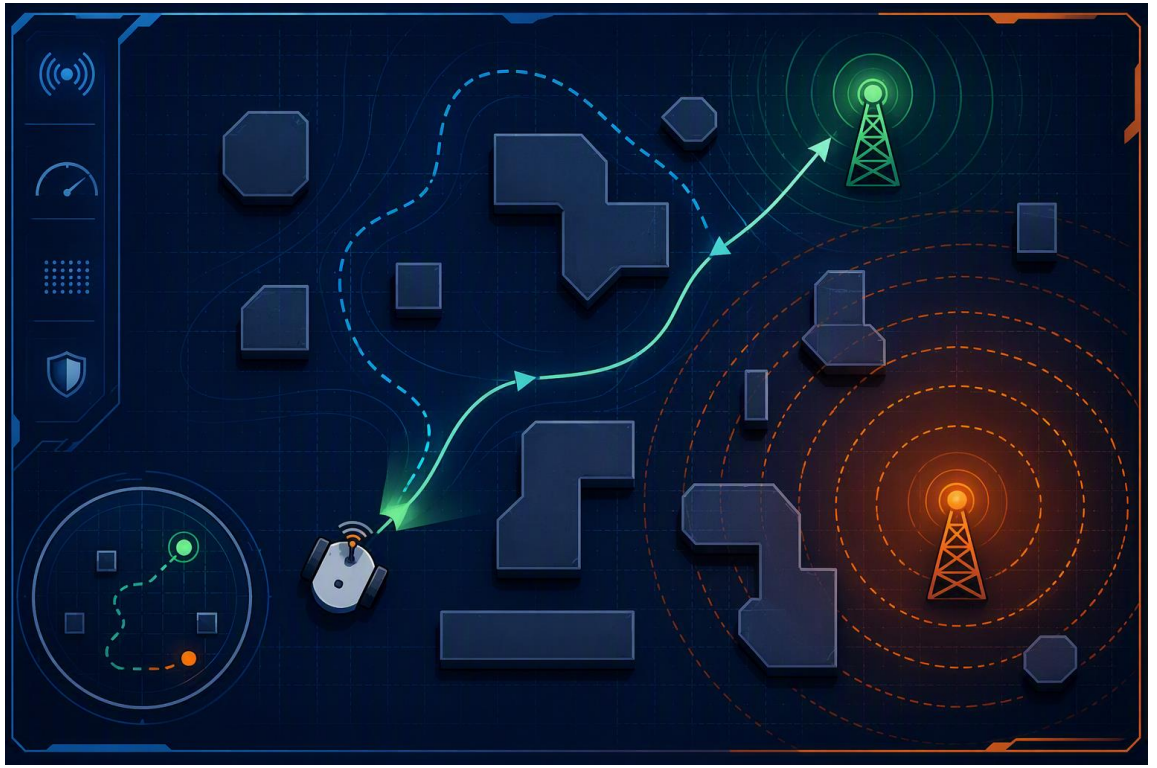
The controller does not use robot or target coordinates.

NO PRELOADED MAP

Obstacles are discovered only through local contact.

LOW-DIMENSIONAL FEEDBACK

Navigation is driven by a scalar signal intensity.



The Robot's Limited Sensors



Touch Sensor returns 1 if the robot is physically touching an obstacle boundary, and 0 otherwise. It does not measure distance or map the shape of the obstacle.



Intensity strength sensor: Reads the current scalar strength of the signal. The robot uses this to determine if it's getting closer to the tower and can detect local maximums by comparing recent samples.



Gradient Alignment Sensor: Indicates whether the robot is facing the direction of the steepest ascent of the signal. Required for asymmetric environments where a straight-line motion in the gradient direction won't point directly to the tower.

The Symmetric Case

The paper defines a signal mapping function from the environment to a scalar value between 0 and 1.
In the perfectly radially symmetric case, the signal decays uniformly in all directions based on distance. The level sets (lines of equal intensity) form concentric circles around the tower.

$$m(p) = \frac{1}{\| p \|}$$



Baseline: Intensity-Based Navigation

Original I-Bug uses local signal information and contact sensing




```julia
function u_ori!(robot::Robot, env::SimulationEnv, step::Int=0)
    dx = env.tower.x - robot.pos.x
    dy = env.tower.y - robot.pos.y
    robot.angle = normalize_angle(atan(dy, dx))
end

function u_fwd!(robot::Robot, env::SimulationEnv)
    nx = robot.pos.x + STEP_SIZE * cos(robot.angle)
    ny = robot.pos.y + STEP_SIZE * sin(robot.angle)
    new_pos = Vec2(nx, ny)
    if is_valid_pos(new_pos, env)
        robot.pos = new_pos
        return true
    end
    return false
end
```

Source: CoreLogic.jl, lines 170–186

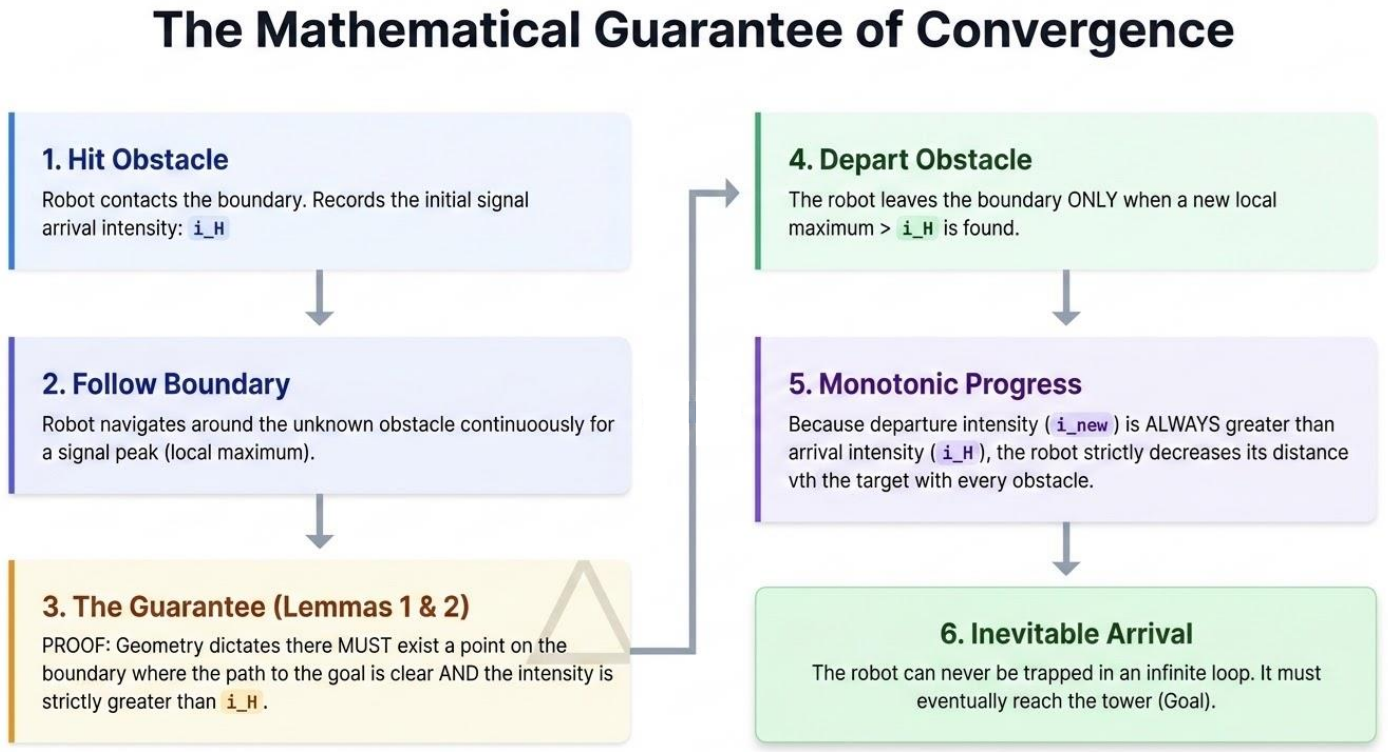
Global Guarantees

Why does this always work? The mathematical proof relies on monotonic progress.

- The robot ONLY leaves an obstacle when it finds a local maximum that is strictly closer to the tower than where it first hit the wall (because current intensity > i_H).

- It cannot get trapped in infinite loops. Because it continually leaves boundaries from closer points, it establishes a strictly increasing sequence of leave points.

- This guarantees it will eventually spiral into the goal, even in highly complex labyrinths.



The Assumption We Break

Original progress logic assumes that measured intensity belongs to the real target

Original setting

One target signal

Intensity indicates real progress

Stored maxima remain meaningful



Adversarial setting

A second source adds false intensity

Temporary peaks imitate progress

Stored thresholds can be corrupted

Question: can robustness be added without localization or mapping?

Adversarial Signal Model

Real and false sources are superimposed at each simulation step

$$I_{perceived}(p,k) = I_{real}(p) + a(t) \cdot I_{enemy}(p)$$

REAL SOURCE

Navigation target
Always active

ACTIVATION a(t)

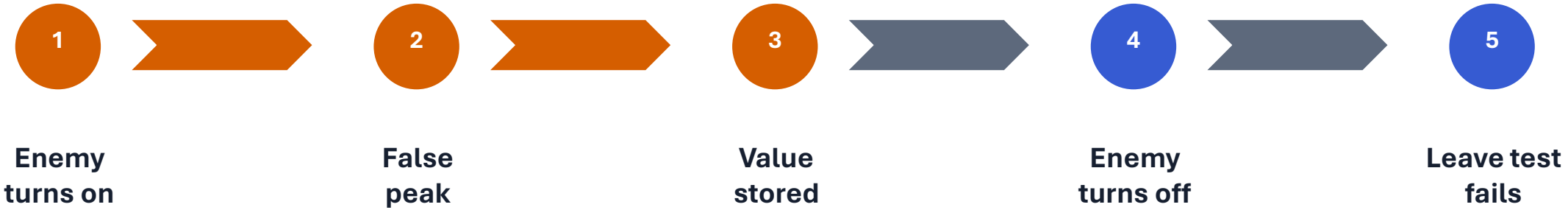
Binary on/off schedule
Starts after step 20

ENEMY SOURCE

False tower
Strength factor = 1.0

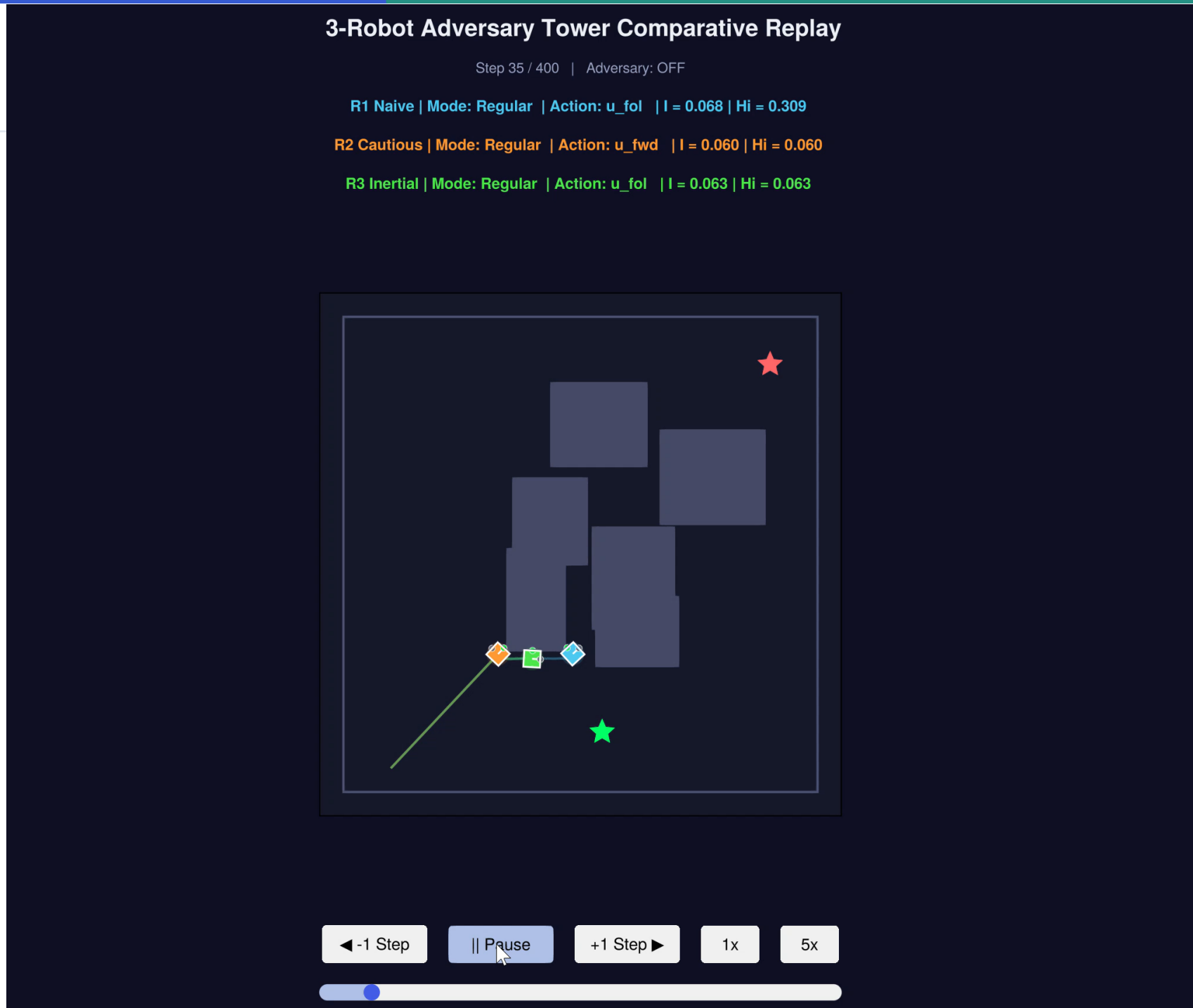
Failure Mechanism

A short false peak can create a long-lasting navigation error



The physical environment is unchanged. The attack targets the controller's interpretation of progress.

Failure Mechanism



Research Question and Contributions

Can I-Bug tolerate intermittent false intensity using only limited controller memory?

01

Threat model

Intermittent false tower

02

Controllers

Cautious and Inertial

03

Toolchain

Julia simulation and replay

04

Evidence

100,000 paired environments

Three Controller Responses

Same environment, same signal, different reaction to detected interference

NAIVE

Trust the perceived signal

No anomaly response

Baseline controller

CAUTIOUS

Pause on a spike

Protect controller state

Resume after confirmation

INERTIAL

Persist with valid behavior

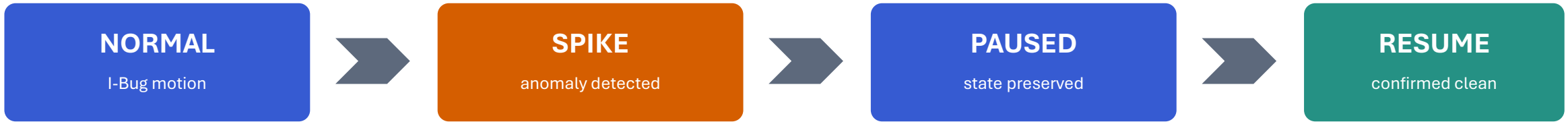
Protect controller state

Continue making progress

All strategies retain local contact protection.

Strategy 2: Cautious

Protect state by pausing during suspected interference



PROTECTION

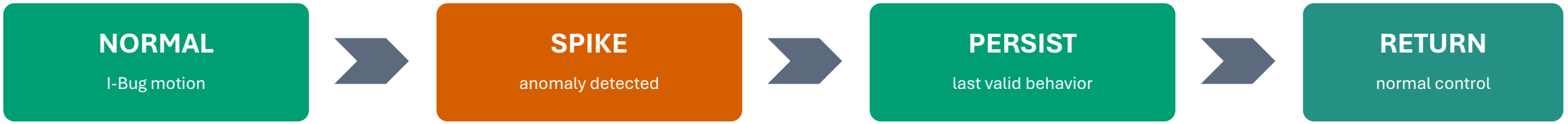
Do not update intensity-dependent memory from suspected corrupted readings.

COST

Waiting improves protection but adds navigation time.

Strategy 3: Inertial

Protect state while preserving the last valid navigation behavior



PROTECTION

Corrupted readings do not overwrite the protected controller state.

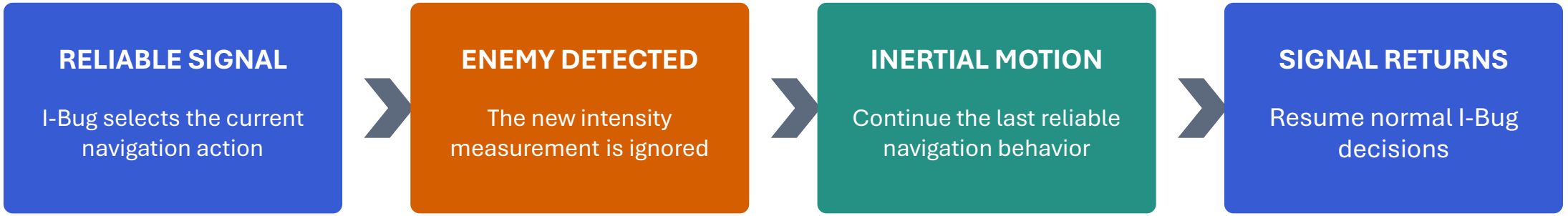
ADVANTAGE

The robot continues useful motion instead of waiting.

“Inertial” means action persistence, not physical momentum.

The Idea Behind the Inertial Strategy

When the signal becomes unreliable, trust the last reliable action instead of the new measurement



The intuition is similar to inertia

A moving system does not immediately change direction because of one suspicious observation. It continues the previously selected behavior until reliable sensing returns.

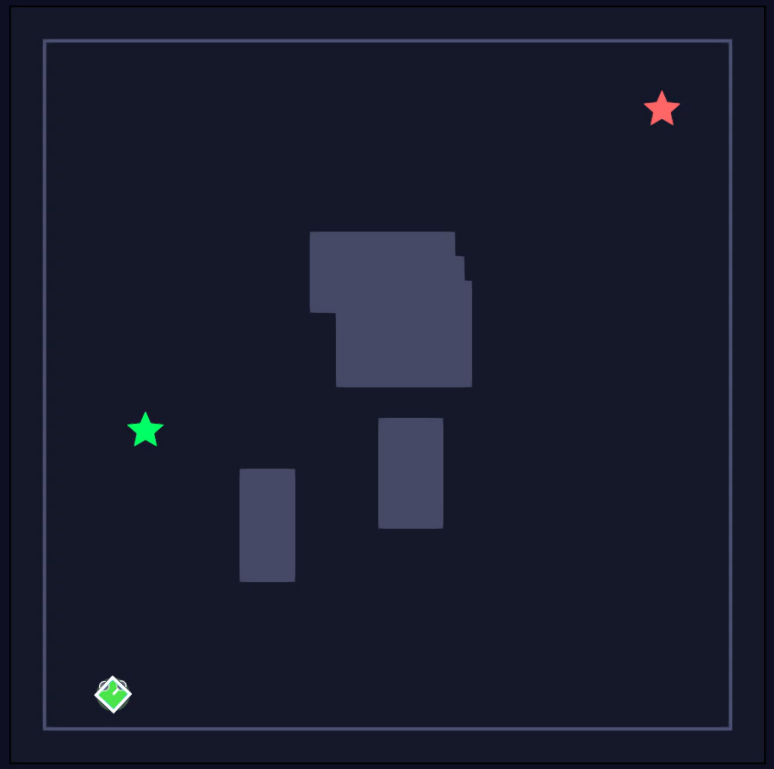
3-Robot Adversary Tower Comparative Replay

Step 0 / 400 | Adversary: OFF

R1 Naive | Mode: Regular | Action: u_ori | I = 0.043 | Hi = 0.043

R2 Cautious | Mode: Regular | Action: u_ori | I = 0.043 | Hi = 0.043

R3 Inertial | Mode: Regular | Action: u_ori | I = 0.043 | Hi = 0.043



◀ -1 Step | > Play | +1 Step ▶ | 1x | 5x

Implementation Architecture

Modular pipeline from world generation to statistical analysis



INTERACTIVE PATH

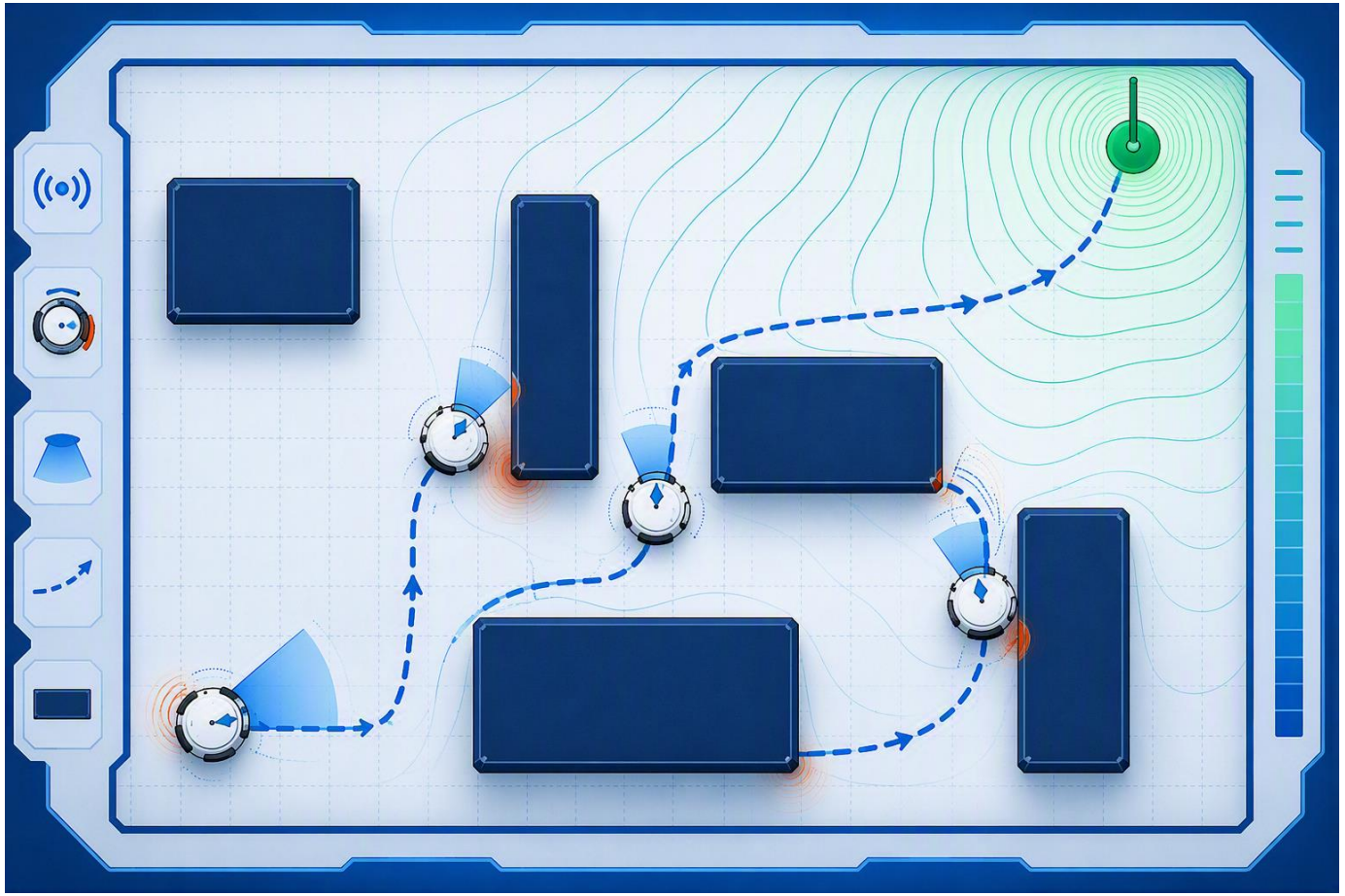
Viewer supports trajectory inspection and deterministic replay.

BATCH PATH

Logger evaluates all strategies and writes one row per result.

Simulation Environment

The same generated world is reused across all three strategies



WORLD

20 × 20

OBSTACLES

3 to 9 rectangles

START → TARGET

(2, 1) → (18, 18)

ROBOT

Move step 0.3 | radius 0.3

Statistical Evaluation

Final batch experiment and analysis protocol

ENVIRONMENTS

100,000
Randomly generated

STRATEGIES

3
Same environment and seed

TOTAL RECORDS

300,000
One result per strategy

STEP BUDGET

400
Failure if target not reached

- Success rate within 400 steps
- Steps for successful runs only
- Paired comparisons by RunID and seed

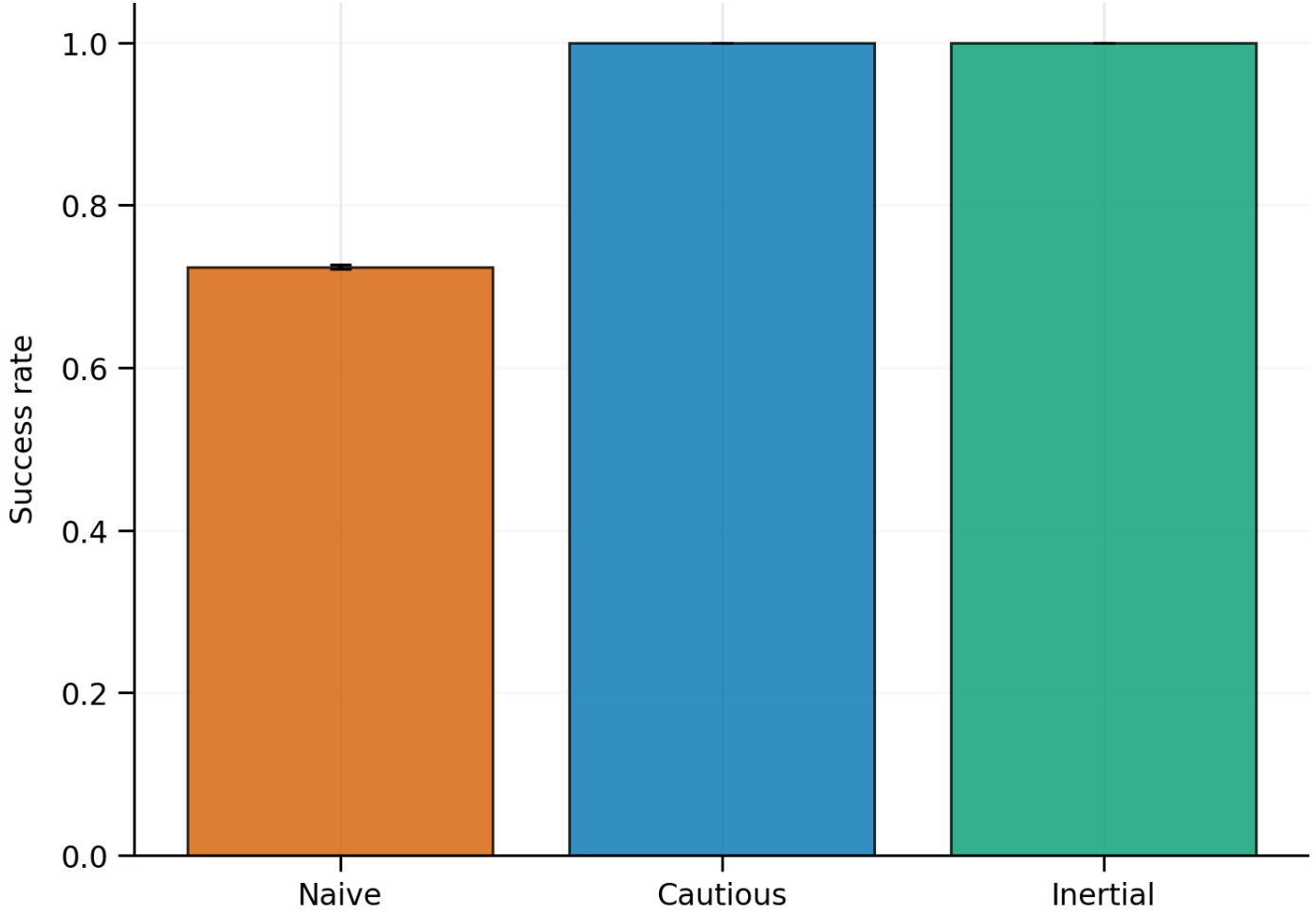
WHY PAIRED?

Every strategy faces the same obstacles, tower placement, and random seed.

A failed run is not a 400-step arrival.

Navigation Reliability

Success within the 400-step budget



NAIVE

72.426% | 27,574 failures

CAUTIOUS

99.964% | 36 failures

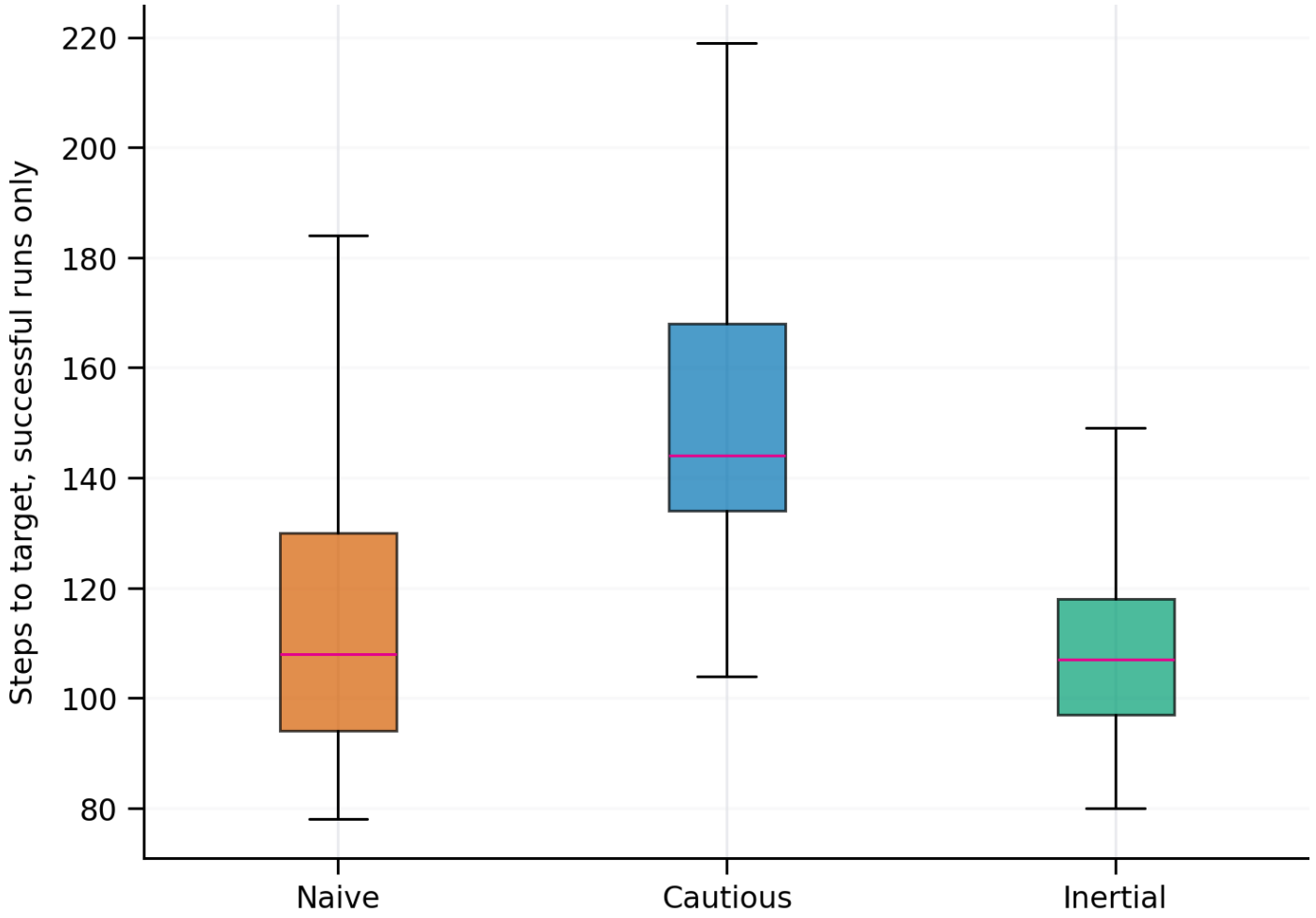
INERTIAL

99.969% | 31 failures

Both extensions remove almost all observed failures.

Navigation Efficiency

Steps to target, successful runs only



MEAN / MEDIAN

Naive	122.21 / 108
Cautious	151.55 / 144
Inertial	109.03 / 107

Cautious pays a clear waiting-time cost.

Inertial preserves robustness with the lowest mean.

100,000 matched environments | 300,000 strategy runs

Cautious vs. Inertial

Direct paired comparison in identical environments

BOTH SUCCEEDED

99,964

INERTIAL-ONLY SUCCESS

5

BOTH FAILED

31

EFFICIENCY WHEN BOTH SUCCEEDED

Inertial was faster in all 99,964 paired successes.

42.53 fewer steps on average | 38-step median reduction

RELIABILITY DIFFERENCE

$p = 0.0625$

Not significant at $\alpha = 0.05$

Limitations and Validity

Results are strong evidence for the evaluated model, not a universal proof

MODEL

Radially symmetric fields

Periodic enemy activation

Axis-aligned rectangles

SIMULATION

Discrete translation and rotation

Idealized sensing

400-step budget

GENERALIZATION

One environment distribution

No physical robot test

No universal convergence proof

Claims are limited to the implemented model and sampled environments.

Conclusions

01

Adversarial intensity harms the Naive controller.

Success falls to 72.426%.

02

Both extensions achieve approximately 99.96% success.

Only 31 to 36 failures in 100,000 runs.

03

Cautious is robust but slower.

Mean successful path: 151.55 steps.

04

Inertial gives the best observed trade-off.

Comparable reliability with much lower delay.

Empirical conclusion under the evaluated simulation model.

THANK YOU

Questions?

Adversarially Robust I-Bug Navigation

Avraham Guez | Tsuriel Vizel

Main takeaway: limited controller memory can protect minimalist navigation from intermittent false signals.

